

APP Mapping and Navigation

APP Mapping and Navigation

1. Course Content
2. Preparations
3. Start the Agent
4. APP Mapping Example
5. APP Navigation Example
6. Source Code Analysis
 - 6.1 Launch File

1. Course Content

Control the car to perform mapping and navigation using the mobile app "ROS Robot".

2. Preparations

1. First, you need to download the "ROS Robot" app on your phone. Android/iOS users can scan the QR code to download the remote control software. iOS users can also search for and download the mapping and navigation app "ROSRobot" from the App Store.



2. The car and the phone must be on the same local network, which can be achieved by connecting to the same Wi-Fi.

3. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
mircoROS_Agent
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1765183764.822476] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1765183764.826313] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x006(2), participant_
id: 0x000(1)
[1765183764.829482] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1765183764.834154] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1765183764.838233] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x007(2), participant_
id: 0x000(1)
[1765183764.842504] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1765183764.847468] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

4. APP Mapping Example

- Enter the following command in the car's terminal to start the car's APP mapping service node. Choose **one** of the following two mapping methods:

[!NOTE]

The launch commands for the different mapping methods are the same here; they are switched using the `slam_mode` launch parameter.

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container

```
exec_m3
```

- Cartographer mapping method:

```
ros2 launch m3_bringup bringup_app.launch.py slam_mode:=cartographer
```

- slam_toolbox mapping method:

```
ros2 launch m3_bringup bringup_app.launch.py slam_mode:=slam_toolbox
```

- Open "ROS Robot", and it will display as shown below.

- Enter the car's IP address -> ROS2 -> videoTopic/camera/rgb/image_raw/compressed -> [Connect]

Enter robot IP

☐ zh ☒ en

Please enter IP address

192.168.11.203|

☐ ROS1

☒ ROS2

Video Topic

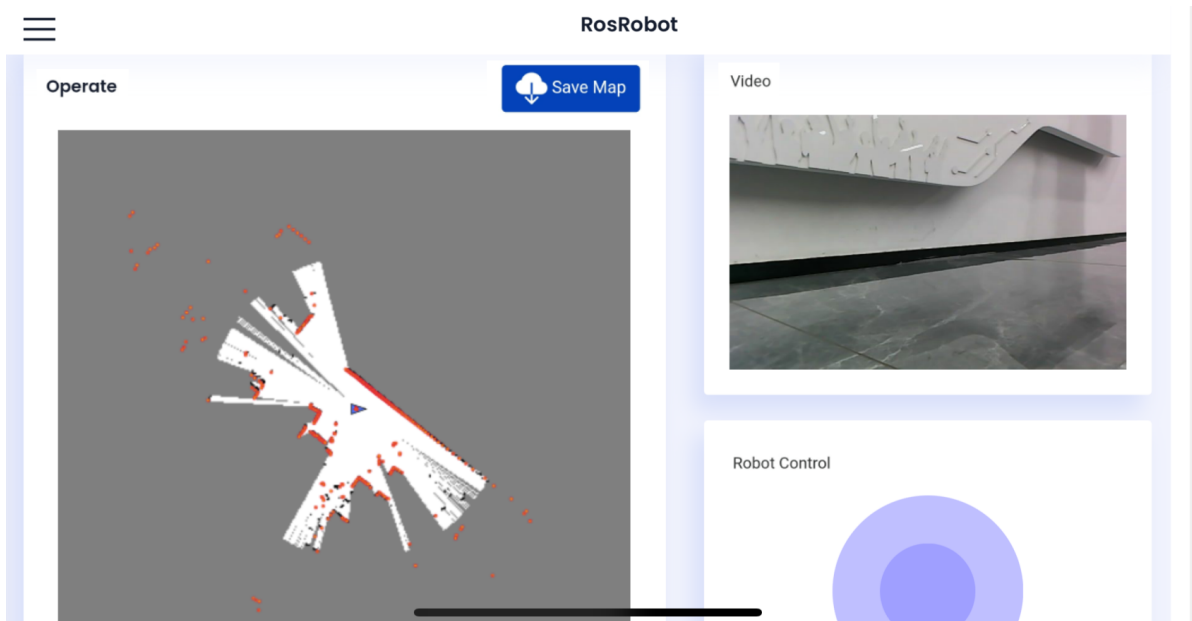
☐ /usb_cam/image_raw/compressed

☒ /camera/rgb/image_raw/compressed

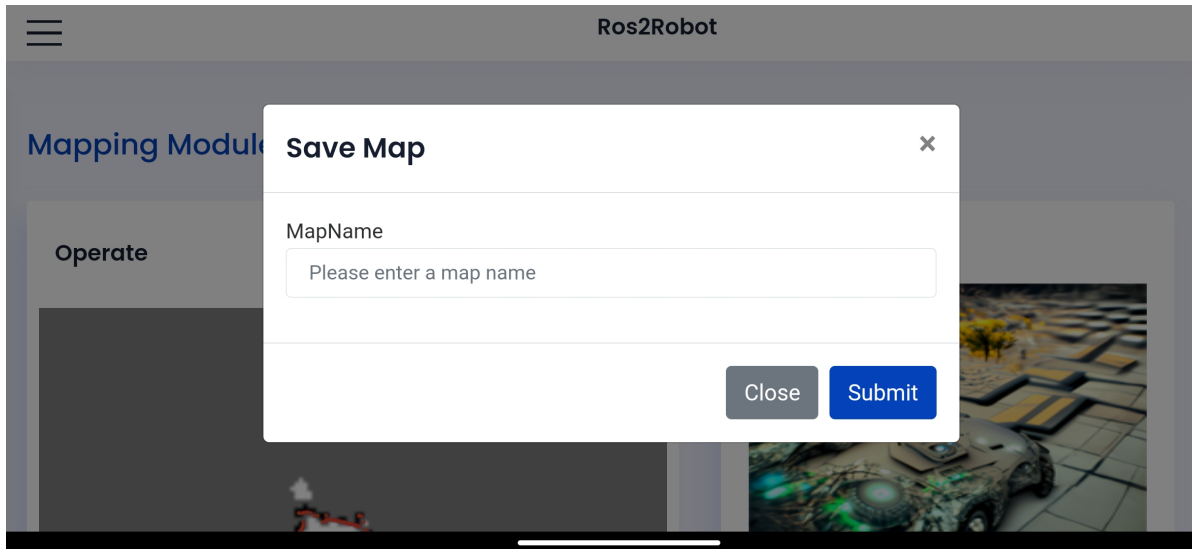
☐ /csi/image_raw/compressed

Connect

- After a successful connection, it will display as follows:



- Use the joystick to slowly move the car to cover the area you need to map, then click "Save Map", enter a map name, and click "Submit" to save the map.



- The map will be automatically saved to the `~/map` directory on the system, and the terminal will indicate that the map has been saved successfully.

```
[server-6] [INFO] [1765334967.910089971] [map_saver]: Creating
[server-6] [INFO] [1765334967.912193848] [map_saver]: Configuring
[server-6] [INFO] [1765334967.922756528] [map_saver]: Saving map from 'map' topic to '/home/jetson/map/4444' file
[server-6] [WARN] [1765334967.922858419] [map_saver]: Free threshold unspecified. Setting it to default value: 0.250000
[server-6] [WARN] [1765334967.922887156] [map_saver]: Occupied threshold unspecified. Setting it to default value: 0.650000
[server-6] [WARN] [1765334967.951984200] [map_io]: Image format unspecified. Setting it to: pgm
[server-6] [INFO] [1765334967.954177999] [map_io]: Received a 449 X 224 map @ 0.05 m/pix
[server-6] [INFO] [1765334968.016817320] [map_io]: Writing map occupancy data to /home/jetson/map/4444.pgm
[server-6] [INFO] [1765334968.019211798] [map_io]: Writing map metadata to /home/jetson/map/4444.yaml
[server-6] [INFO] [1765334968.019438653] [map_io]: Map saved
[server-6] [INFO] [1765334968.019460606] [map_saver]: Map saved successfully
[server-6] [INFO] [1765334968.023690856] [map_saver]: Destroying
```

5. APP Navigation Example

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

```
ros2 launch m3_bringup bringup_app.launch.py navigation_mode:=True
map:=/xxx/map.yaml
```

[!NOTE]

Optional Parameter Descriptions

- `map`

- The path to the map file. You need to enter the absolute path + map yaml name.
The default is `~/map/map.yaml`.

- Open "ROS Robot", and it will display as shown below.
- Enter the car's IP address -> `ROS2` -> `videoTopic/camera/rgb/image_raw/compressed` -> [Connect]

Enter robot IP

☐ zh ☒ en

Please enter IP address

192.168.11.203|

☐ ROS1

☒ ROS2

Video Topic

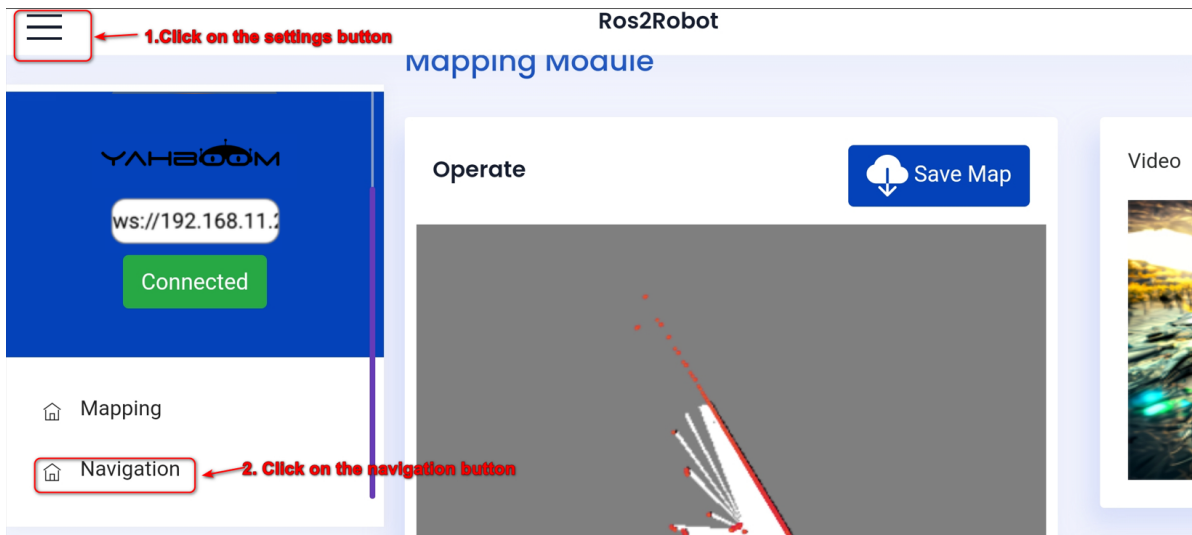
☐ /usb_cam/image_raw/compressed

☒ /camera/rgb/image_raw/compressed

☐ /csi/image_raw/compressed

Connect

- As shown below, select the navigation interface:



- Then, based on the car's actual pose, click "Init Point" to give the car an initial target point. If the Lidar scan area roughly coincides with the actual obstacles, the pose is accurate, as shown below.



- Click "Nav Point" to give the car a destination. The car will plan a path and move to the destination along that path, as shown below:



6. Source Code Analysis

6.1 Launch File

Source code path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/common/bringup_app.launch.py
```

- The key startup program is as follows:
- The program uses the `slam_mode` and `navigation_mode` launch parameters to decide between mapping and navigation modes. It starts a rosbridge WebSocket server to transmit relevant data via the WebSocket protocol.

```
# Start the slam_toolbox mapping mode.
slam_toolbox_launch= GroupAction(
    condition=IfCondition(PythonExpression(["'", slam_mode, "' ==
'slam_toolbox' and '", navigation_mode, "' == 'False' "]])),
    actions=[
        ExecuteProcess(
            cmd=['ros2', 'launch', 'm3_bringup', 'car_base.launch.py'],
            output='screen'
        ),
        Node(
            package='slam_toolbox',
            executable='async_slam_toolbox_node',
            parameters=[async_slam_params_file],
            name='async_slam_toolbox',
            output='screen'
        )
    ]
)
```

```

# Start Cartographer mapping mode.
cartographer_launch=IncludeLaunchDescription(
    PythonLaunchDescriptionSource(os.path.join(m3_bringup_dir,
        'launch',
        'slam',
        'cartographer',
        'cartographer.launch.py'))),
    launch_arguments={
        "gui": "False",
    }.items(),
    condition=IfCondition(PythonExpression(["'", slam_mode, "' ==
'cartographer' and '", navigation_mode, "' == 'False' "]))

)

# Start the rosbridge node.
bringup_rosbridge_server = ExecuteProcess(
    cmd=['ros2', 'launch', 'rosbridge_server',
'rosbridge_websocket_launch.xml'],
    output='screen'
)

# Compressed image publishing node
rgb_image_node = Node(
    package='laserscan_to_point_publisher',
    executable='pub_rgb_image',
    name='publish_rgb_frame',
    parameters=[{'rgb_topic': rgb_topic}],
    output='screen',
)

# Radar point cloud publishing node
laserscan_to_point_publisher= Node(
    package='laserscan_to_point_publisher',
    executable='laserscan_to_point_publisher',
    name='laserscan_to_point_publisher',
    output='screen',
)

# Robot pose publishing node
robot_pose_publisher=Node(
    package="robot_pose_publisher_ros2",
    executable="robot_pose_publisher",
    name="robot_pose_publisher",
    output="screen",
    emulate_tty=True,
    parameters=[
        {"use_sim_time": True},
        {"is_stamped": True},
        {"map_frame": "map"},
        {"base_frame": "base_link"}
    ]
)

# App map saving service node
yahboom_app_save_map_node = Node(
    package='yahboom_app_save_map',
    executable="server",

```



```

        name="yahboom_app_save_map_server",
        output="screen",
    )

    app_nav= GroupAction(
        condition=IfCondition(PythonExpression(["'", navigation_mode, "' ==
'True'" ])),
        actions=[
            ExecuteProcess(
                cmd=['ros2', 'launch', 'm3_bringup', 'car_base.launch.py'],
                output='screen'
            ),
            IncludeLaunchDescription(

PythonLaunchDescriptionSource(os.path.join(get_package_share_directory("nav2_br
ingup"),

                                "launch",
                                'bringup_launch.py')),

            launch_arguments={
                "map": map,
                "use_sim_time": "False",
                "params_file": os.path.join(m3_bringup_dir,
                                            'config',
                                            'navigation_config',
                                            'M3_nav2_params.yaml'
                                            ),
            }.items(),
        )
    ]
)

```